REVISION CHEAT SHEET:

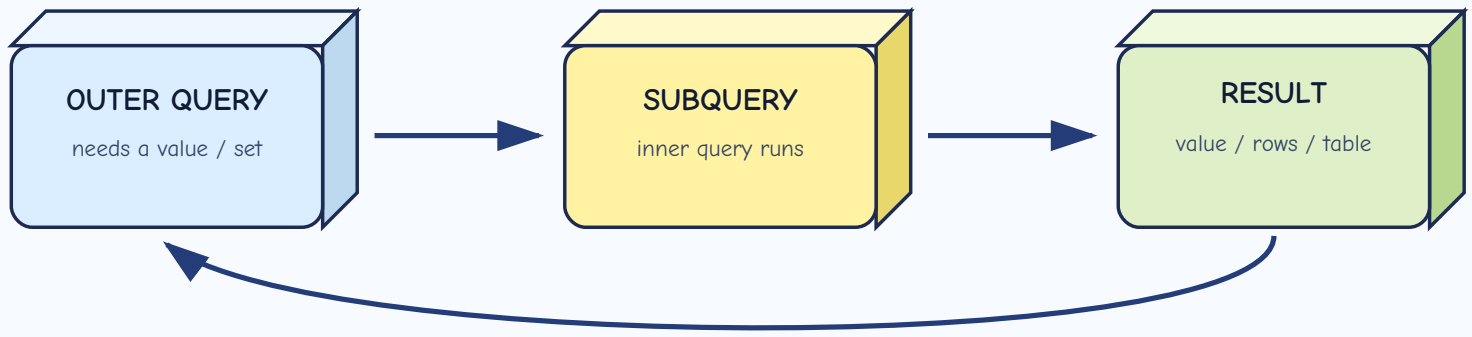
SQL Server Subqueries

INTERVIEW-READY NOTES - SCALAR - CORRELATED - EXISTS - IN - ANY / ALL - DERIVED TABLES

3D DIAGRAM

How a Subquery Feeds the Outer Query

The inner query returns information that the outer query consumes.



Outer query continues using the returned shape.

Always identify the shape returned by the inner query before choosing =, IN, EXISTS, ANY, ALL, or a derived-table pattern.

TOPIC & SUBTOPIC INDEX

Study order: fundamentals -> result shapes -> correlated vs non-correlated -> SELECT/FROM/WHERE/HAVING locations -> IN/EXISTS/ANY/ALL -> nesting and rewrites -> performance -> practical patterns -> interview revision.

Why Use a Subquery?	First Simple Example	Outer Query vs Inner Query
Main Ways to Classify Subqueries	Scalar Subquery	Single-Row Subquery
TOP 1 Subquery	ORDER BY Inside a Subquery	Multi-Row Subquery
Multi-Column Subquery	Non-Correlated Subquery	Correlated Subquery
Subquery in SELECT	Subquery in FROM	Subquery in WHERE
Subquery in HAVING	Subquery with JOIN	IN with Subquery
NOT IN with Subquery	EXISTS	NOT EXISTS
EXISTS vs IN	NOT EXISTS vs NOT IN	ANY
SOME	ALL	ANY vs ALL
Comparison Operators with Subqueries	Nested Subqueries	Correlated Subquery Example — Gifts Above Donor Average
Converting Subqueries to JOINs	Subquery Performance Considerations	Subquery Rules Summary
Practical Pattern 1 — Gifts Above Overall Average	Subquery vs CTE	Subquery vs Derived Table
Subquery vs JOIN	Common Mistakes	Master Subquery Decision Guide
Master Classification Table	Location Summary	Operator Summary
25 Interview Questions and Query Problems	Final Revision Map	Final Easy Memory Rules

A **subquery** is a SQL query written inside another SQL statement.

It is sometimes called:

EXAMPLE / FLOW

```
Inner Query
Nested Query
```

The query containing it is called the:

EXAMPLE / FLOW

```
Outer Query
```

The simplest idea is:

EXAMPLE / FLOW

```
Outer Query
     |
     | needs some information
     v
Subquery
     |
     | returns value / rows
     v
Outer Query continues
```

A subquery can return:

EXAMPLE / FLOW

```
One value

One row

Multiple rows

Multiple columns

Multiple rows and multiple columns
```

How many rows and columns the subquery returns determines **where and how you are allowed to use it**.

Microsoft documents three important practical subquery forms: subqueries used with `IN` or `ANY/ALL`, subqueries used with an ordinary comparison operator that must return one value, and existence tests using `EXISTS`. SQL Server also supports nested subqueries, with up to 32 nesting levels theoretically possible, although the practical limit can be lower depending on query complexity and memory.

FundraisingDb Tables Used in This Chapter

We will use:

EXAMPLE / FLOW

```
constituent.Constituent

fundraising.Gift

fundraising.Campaign

fundraising.Fund

fundraising.GiftAllocation

engagement.Interaction

engagement.Task

core.AppUser
```

The main relationships are:

EXAMPLE / FLOW

```
Constituent
    |
    | 1:M
    v
Gift
    |
    +----------> Campaign
    |
    v
GiftAllocation
    |
    v
Fund
```

and:

EXAMPLE / FLOW

```
Constituent
    |
    +---- Interaction
    |
    +---- Task

AppUser
    |
    +---- Interaction
    |
    +---- Task
```

Why Use a Subquery?

A subquery is useful when one query needs the result of another query.

Examples:

EXAMPLE / FLOW

```
Find Gifts larger than the average Gift.

Find Constituents who have at least one Gift.

Find Constituents who have never made a Gift.

Find Campaigns whose total raised amount exceeds
the average Campaign total.

Find the latest Gift amount for every Constituent.

Find Funds that appear in GiftAllocation.

Compare a Gift amount against all Gifts in a Campaign.
```

First Simple Example

Business requirement:

Return Gifts larger than the average Gift amount.

First calculate the average:

SQL

```sql
SELECT
    AVG(TotalAmount)
FROM fundraising.Gift;
```

Suppose the result is:

EXAMPLE / FLOW

```
$850
```

Now compare every Gift against that value.

SQL

```
SELECT
    GiftId,
    ConstituentId,
    GiftDate,
    TotalAmount
FROM fundraising.Gift
WHERE TotalAmount >
(
    SELECT
        AVG(TotalAmount)
    FROM fundraising.Gift
);
```

The inner query:

SQL

```
SELECT AVG(TotalAmount)
FROM fundraising.Gift
```

returns:

EXAMPLE / FLOW

```
one value
```

The outer query then compares each Gift to that value.

Subquery Flow

EXAMPLE / FLOW

```
fundraising.Gift
        |
        v
Calculate AVG(TotalAmount)
        |
        v
Example result = $850
        |
        v
Outer Gift query
        |
        v
Keep Gifts > $850
```

Outer Query vs Inner Query

Consider:

SQL

```
SELECT
    GiftId,
    TotalAmount
FROM fundraising.Gift
WHERE TotalAmount >
(
    SELECT AVG(TotalAmount)
    FROM fundraising.Gift
);
```

Outer query:

SQL

```
SELECT
    GiftId,
```

```
    TotalAmount
FROM fundraising.Gift
```

Inner query:

SQL

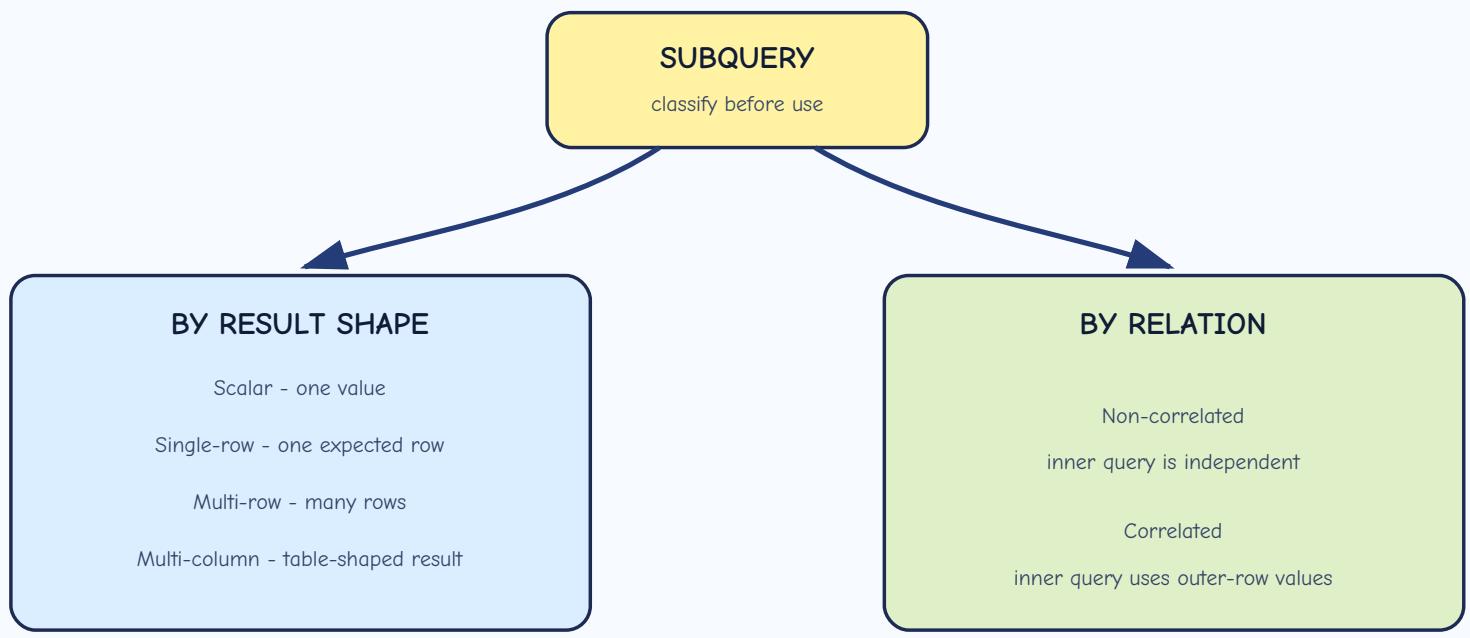
```
SELECT AVG(TotalAmount)
FROM fundraising.Gift
```

The outer query uses the inner query's result.

Main Ways to Classify Subqueries

3D DIAGRAM

Subquery Classification Map



Result shape determines where the subquery can be used; correlation determines whether the inner logic depends on the current outer row.

Subqueries can be classified in several ways.

By the result they return:

EXAMPLE / FLOW

```
Scalar subquery

Single-row subquery

Multi-row subquery

Multi-column subquery
```

By how they relate to the outer query:

EXAMPLE / FLOW

```
Non-correlated subquery

Correlated subquery
```

By where they appear:

EXAMPLE / FLOW

```
SELECT

FROM
```

```
WHERE

HAVING

JOIN / ON

INSERT / UPDATE / DELETE
```

Scalar Subquery

Simple Definition

A **scalar subquery** returns exactly one value.

That means:

EXAMPLE / FLOW

```
At most one row
and
one column
```

A scalar subquery can normally be used anywhere SQL Server expects a single expression.

Example:

SQL

```sql
SELECT
    GiftId,
    TotalAmount,

    (
        SELECT AVG(TotalAmount)
        FROM fundraising.Gift
    ) AS OverallAverageGift

FROM fundraising.Gift;
```

Result Conceptually

Suppose:

EXAMPLE / FLOW

```
Overall average = $850
```

Then:

GiftId	TotalAmount	OverallAverageGift
10000	1000	850
10001	500	850
10002	1500	850

The scalar value is repeated for each output row.

Scalar Subquery Rule

A scalar subquery must not return more than one row.

This is valid:

SQL

```sql
WHERE TotalAmount =
(
    SELECT MAX(TotalAmount)
    FROM fundraising.Gift
);
```

`MAX()` produces one value.

This may fail:

```sql
WHERE CampaignId =
(
    SELECT CampaignId
    FROM fundraising.Campaign
);
```

because the inner query may return:

```
many CampaignIds
```

and = expects one value.

SQL Server raises an error when an unmodified comparison operator such as =, >, <, >=, or <= is used with a subquery that returns more than one value.

Single-Row Subquery

Simple Definition

A **single-row subquery** is expected to return one row.

In practical SQL Server comparisons, the most common form returns:

```
one row
+
one column
```

Example:

Find Gifts made to the Campaign having code `CANCER2026`.

```sql
SELECT
    GiftId,
    ConstituentId,
    CampaignId,
    TotalAmount
FROM fundraising.Gift
WHERE CampaignId =
(
    SELECT CampaignId
    FROM fundraising.Campaign
    WHERE CampaignCode = 'CANCER2026'
);
```

This assumes:

```
CampaignCode
```

is unique.

Important Single-Row Rule

Do not use:

```
=
```

simply because you expect the query to return one row.

The database must actually guarantee or logically ensure it.

Good situations include:

```
Primary key lookup

Unique key lookup
```

```
Aggregate such as MAX / MIN / AVG / SUM / COUNT

TOP (1) with deterministic ORDER BY
```

TOP 1 Subquery

Requirement:

> Return Gifts larger than the latest Gift amount in Campaign 200.

First find the latest Gift:

SQL

```
SELECT TOP (1)
    TotalAmount
FROM fundraising.Gift
WHERE CampaignId = 200
ORDER BY
    GiftDate DESC,
    GiftId DESC;
```

Then:

SQL

```
SELECT
    GiftId,
    GiftDate,
    TotalAmount
FROM fundraising.Gift
WHERE TotalAmount >
(
    SELECT TOP (1)
        TotalAmount
    FROM fundraising.Gift
    WHERE CampaignId = 200
    ORDER BY
        GiftDate DESC,
        GiftId DESC
);
```

The tie-breaker:

EXAMPLE / FLOW

```
GiftId DESC
```

makes the `TOP (1)` selection deterministic when multiple Gifts share the same `GiftDate`.

ORDER BY Inside a Subquery

A normal `ORDER BY` is not valid inside a SQL Server subquery unless it participates in constructs such as `TOP` or `OFFSET/FETCH`. Even then, the ordering is used to determine which rows are selected; it does not guarantee outer-query ordering.

Therefore this is not normally valid:

SQL

```
SELECT *
FROM fundraising.Gift
WHERE GiftId IN
(
    SELECT GiftId
    FROM fundraising.Gift
    ORDER BY GiftDate
);
```

But this is valid:

SQL

```
SELECT *
FROM fundraising.Gift
WHERE GiftId =
```

```
(
    SELECT TOP (1)
        GiftId
    FROM fundraising.Gift
    ORDER BY
        GiftDate DESC,
        GiftId DESC
);
```

Multi-Row Subquery

Simple Definition

A **multi-row subquery** returns more than one row.

Example:

SQL

```
SELECT
    CampaignId
FROM fundraising.Campaign
WHERE Status = 'Active';
```

might return:

EXAMPLE / FLOW

```
200
201
205
210
```

You cannot normally compare that directly with:

SQL

```
=
```

Instead, use operators designed for multiple values:

EXAMPLE / FLOW

```
IN

NOT IN

ANY

SOME

ALL

EXISTS

NOT EXISTS
```

Multi-Row Example with IN

Requirement:

Return Gifts belonging to active Campaigns.

SQL

```
SELECT
    GiftId,
    ConstituentId,
    CampaignId,
    TotalAmount
FROM fundraising.Gift
WHERE CampaignId IN
(
    SELECT CampaignId
```

```
    FROM fundraising.Campaign
    WHERE Status = 'Active'
);
```

The subquery can return many CampaignIds.

IN checks whether each Gift's CampaignId matches any returned value.

Multi-Column Subquery

Simple Definition

A **multi-column subquery** returns more than one column.

Example:

SQL

```
SELECT
    ConstituentId,
    COUNT(*) AS GiftCount,
    SUM(TotalAmount) AS LifetimeGiving
FROM fundraising.Gift
GROUP BY ConstituentId;
```

This returns:

EXAMPLE / FLOW

```
ConstituentId
GiftCount
LifetimeGiving
```

A multi-column result cannot be used as a scalar expression.

It is commonly used as:

EXAMPLE / FLOW

```
a derived table in FROM

a joined derived table

an EXISTS subquery where the selected values themselves are irrelevant
```

Multi-Column Subquery in FROM

SQL

```
SELECT
    c.ConstituentId,
    c.DisplayName,
    gs.GiftCount,
    gs.LifetimeGiving

FROM constituent.Constituent AS c

INNER JOIN
(
    SELECT
        ConstituentId,
        COUNT(*) AS GiftCount,
        SUM(TotalAmount) AS LifetimeGiving

    FROM fundraising.Gift

    GROUP BY ConstituentId
) AS gs
    ON gs.ConstituentId =
       c.ConstituentId;
```

The subquery returns multiple rows and multiple columns.

Because it is placed in the **FROM** clause, SQL Server treats it as a:

EXAMPLE / FLOW

```
derived table
```

Important SQL Server Multi-Column Note

SQL Server does not use PostgreSQL-style tuple comparison syntax such as:

EXAMPLE / FLOW

```
(ConstituentId, AppUserId)
IN
(
   SELECT ...
)
```

for ordinary row-value comparison.

When you need to compare multiple columns, a common SQL Server pattern is:

EXAMPLE / FLOW

```
EXISTS
```

with multiple correlated predicates.

Example:

SQL

```
SELECT
    t.TaskId,
    t.ConstituentId,
    t.AssignedToUserId
FROM engagement.Task AS t

WHERE EXISTS
(
    SELECT 1
    FROM engagement.Interaction AS i

    WHERE i.ConstituentId =
          t.ConstituentId

      AND i.FundraiserUserId =
          t.AssignedToUserId
);
```

This asks:

Does this exact Constituent/AppUser pair also appear in Interaction?

Non-Correlated Subquery

Simple Definition

A **non-correlated subquery** does not reference columns from the outer query.

It can conceptually run independently.

Example:

SQL

```
SELECT
    GiftId,
    TotalAmount
FROM fundraising.Gift
WHERE TotalAmount >
(
    SELECT AVG(TotalAmount)
    FROM fundraising.Gift
);
```

The inner query:

SQL

```
SELECT AVG(TotalAmount)
FROM fundraising.Gift
```

does not depend on the current outer Gift.

Non-Correlated Flow

EXAMPLE / FLOW

```
Subquery
Calculate overall average Gift
        |
        v
Example = $850
        |
        v
Outer query compares
every Gift to $850
```

The same inner result is conceptually used for the whole outer query.

Correlated Subquery

Simple Definition

A **correlated subquery** references a value from the outer query.

It cannot be understood independently because the inner query depends on the current outer row.

Example:

Show each Gift and the total amount allocated for that Gift.

SQL

```
SELECT
    g.GiftId,
    g.TotalAmount,

    (
        SELECT
            SUM(ga.AllocationAmount)

        FROM fundraising.GiftAllocation AS ga

        WHERE ga.GiftId =
              g.GiftId
    ) AS AllocationTotal

FROM fundraising.Gift AS g;
```

The subquery contains:

SQL

```
ga.GiftId = g.GiftId
```

`g.GiftId` comes from the outer query.

Therefore the subquery is correlated.

Correlated Flow

Suppose outer query reaches:

EXAMPLE / FLOW

```
GiftId = 10000
```

The inner query becomes conceptually:

EXAMPLE / FLOW

```
Sum allocations
where GiftId = 10000
```

Then outer query reaches:

EXAMPLE / FLOW

```
GiftId = 10001
```

The inner query becomes conceptually:

EXAMPLE / FLOW

```
Sum allocations
where GiftId = 10001
```

Flow:

EXAMPLE / FLOW

```
Outer Gift 10000
      |
      v
Find allocations for Gift 10000
      |
      v
Return total

Outer Gift 10001
      |
      v
Find allocations for Gift 10001
      |
      v
Return total
```

Important Performance Clarification

A correlated subquery is logically evaluated relative to each outer row.

That does **not always mean** SQL Server physically executes the inner query once for every row exactly as written.

The optimizer may transform or decorrelate the query and choose joins, semi joins, aggregates, spools, or other plan strategies.

Therefore:

> Do not assume every correlated subquery is automatically slow.

But correlated subqueries can become expensive when the optimizer cannot efficiently transform them, especially when the inner lookup is expensive or poorly indexed.

Correlated vs Non-Correlated

Non-Correlated	Correlated
Independent of outer row	References outer row
Can conceptually run alone	Depends on outer query
Same result may serve all rows	Result may differ per outer row
Often simple aggregate/list	Common with `EXISTS`, scalar lookups
Usually easier to reason about	Can be more expensive if poorly indexed

Subquery in SELECT

A subquery can appear in the `SELECT` list when it returns a single scalar value.

Example:

SQL

```
SELECT
    g.GiftId,
    g.TotalAmount,

    (
        SELECT
            SUM(ga.AllocationAmount)

        FROM fundraising.GiftAllocation AS ga
```

```
        WHERE ga.GiftId = g.GiftId
    ) AS AllocatedAmount

FROM fundraising.Gift AS g;
```

Why Use SELECT Subquery?

Useful when you need an additional calculated lookup value for each row.

Examples:

EXAMPLE / FLOW

```
Allocation total for each Gift

Latest Interaction date for each Constituent

Task count for each AppUser

Largest Gift for each Constituent
```

Latest Interaction Date with SELECT Subquery

SQL

```
SELECT
    c.ConstituentId,
    c.DisplayName,

    (
        SELECT
            MAX(i.InteractionDate)

        FROM engagement.Interaction AS i

        WHERE i.ConstituentId =
              c.ConstituentId
    ) AS LatestInteractionDate

FROM constituent.Constituent AS c;
```

The subquery is:

EXAMPLE / FLOW

```
scalar
+
correlated
```

What If No Matching Rows Exist?

For aggregate functions such as:

EXAMPLE / FLOW

```
SUM

AVG

MAX

MIN
```

the scalar aggregate result can be:

EXAMPLE / FLOW

```
NULL
```

when there are no matching input rows.

Example:

SQL

```
SELECT
    g.GiftId,

    COALESCE
    (
        (
            SELECT
                SUM(ga.AllocationAmount)

            FROM fundraising.GiftAllocation AS ga

            WHERE ga.GiftId = g.GiftId
        ),
        0
    ) AS AllocationTotal

FROM fundraising.Gift AS g;
```

Now Gifts without allocations display:

EXAMPLE / FLOW

```
0
```

instead of:

EXAMPLE / FLOW

```
NULL
```

if zero matches the business meaning.

Subquery in FROM

A subquery in **FROM** is called a:

EXAMPLE / FLOW

```
Derived Table
```

Example:

SQL

```
SELECT
    c.ConstituentId,
    c.DisplayName,
    x.LifetimeGiving

FROM constituent.Constituent AS c

INNER JOIN
(
    SELECT
        ConstituentId,
        SUM(TotalAmount)
            AS LifetimeGiving

    FROM fundraising.Gift

    GROUP BY ConstituentId
) AS x
    ON x.ConstituentId =
       c.ConstituentId;
```

Derived Table Rule

A derived table must have an alias.

Correct:

SQL

```
FROM
(
    SELECT ...
) AS x
```

Do not leave it unnamed.

Why Use a Derived Table?

Use a subquery in **FROM** when:

EXAMPLE / FLOW

```
you need multiple rows and columns,

you want to aggregate before joining,

you want to transform data into a temporary relational result,

or the derived result makes the outer query easier to understand.
```

Subquery in WHERE

This is one of the most common locations.

Example:

Gifts larger than the overall average.

SQL

```
SELECT
    GiftId,
    TotalAmount
FROM fundraising.Gift
WHERE TotalAmount >
(
    SELECT AVG(TotalAmount)
    FROM fundraising.Gift
);
```

Other common WHERE patterns:

EXAMPLE / FLOW

```
= (subquery)

> (subquery)

IN (subquery)

NOT IN (subquery)

EXISTS (subquery)

NOT EXISTS (subquery)

> ANY (subquery)

> ALL (subquery)
```

Subquery in HAVING

A subquery can also be used inside **HAVING**.

Business requirement:

Return Campaigns whose total giving is greater than the average Campaign total.

First we need each Campaign's total.

Then compare it to the average of Campaign totals.

SQL

```
SELECT
    g.CampaignId,
    SUM(g.TotalAmount) AS CampaignTotal
```

```
FROM fundraising.Gift AS g

WHERE g.CampaignId IS NOT NULL

GROUP BY g.CampaignId

HAVING SUM(g.TotalAmount) >
(
    SELECT
        AVG(x.CampaignTotal)

    FROM
    (
        SELECT
            SUM(g2.TotalAmount)
                AS CampaignTotal

        FROM fundraising.Gift AS g2

        WHERE g2.CampaignId
              IS NOT NULL

        GROUP BY g2.CampaignId
    ) AS x
);
```

Understand the HAVING Example

Inner-most query:

EXAMPLE / FLOW

```
Calculate one total per Campaign.
```

Next subquery:

EXAMPLE / FLOW

```
Calculate average of Campaign totals.
```

Outer query:

EXAMPLE / FLOW

```
Calculate each Campaign total.
```

HAVING:

EXAMPLE / FLOW

```
Keep Campaigns whose total
is greater than the average.
```

Subquery with JOIN

One of the most practical patterns is:

EXAMPLE / FLOW

```
Aggregate in a subquery
then JOIN the result.
```

Example:

SQL

```
SELECT
    c.ConstituentId,
    c.DisplayName,
    gs.GiftCount,
    gs.LifetimeGiving

FROM constituent.Constituent AS c

INNER JOIN
```

```
(
    SELECT
        ConstituentId,
        COUNT(*) AS GiftCount,
        SUM(TotalAmount)
            AS LifetimeGiving

    FROM fundraising.Gift

    GROUP BY ConstituentId
) AS gs

    ON gs.ConstituentId =
       c.ConstituentId;
```

Why Aggregate Before JOIN?

Suppose a Constituent has:

EXAMPLE / FLOW

```
100 Gift rows
```

The subquery reduces those to:

EXAMPLE / FLOW

```
1 summary row
```

before joining to `Constituent`.

This often makes the desired result grain much clearer.

Subquery Inside ON

A subquery can also be part of a join condition, although this is less common and can become harder to read.

Example:

SQL

```
SELECT
    g.GiftId,
    g.TotalAmount,
    c.CampaignName

FROM fundraising.Gift AS g

INNER JOIN fundraising.Campaign AS c
    ON c.CampaignId = g.CampaignId

   AND c.GoalAmount >
       (
           SELECT AVG(GoalAmount)
           FROM fundraising.Campaign
           WHERE GoalAmount IS NOT NULL
       );
```

This joins only Campaigns whose goal exceeds the average Campaign goal.

IN with Subquery

Simple Definition

`IN` asks:



Does this value exist anywhere in the values returned by the subquery?

Syntax:

SQL

```
expression IN
(
    subquery
)
```

The subquery must return one column.

IN Example — Gifts in Active Campaigns

SQL

```
SELECT
    GiftId,
    ConstituentId,
    CampaignId,
    TotalAmount

FROM fundraising.Gift

WHERE CampaignId IN
(
    SELECT CampaignId
    FROM fundraising.Campaign
    WHERE Status = 'Active'
);
```

IN Flow

EXAMPLE / FLOW

```
Subquery returns:

200
201
205

Then outer query becomes conceptually:

CampaignId IN (200,201,205)
```

IN Example — Constituents Who Donated

SQL

```
SELECT
    ConstituentId,
    DisplayName

FROM constituent.Constituent

WHERE ConstituentId IN
(
    SELECT ConstituentId
    FROM fundraising.Gift
);
```

This means:

Return Constituents whose ID appears among Gift rows.

Does Duplicate Data Matter Inside IN?

No.

If the subquery returns:

EXAMPLE / FLOW

```
1001
1001
1001
1002
```

IN still asks only:

EXAMPLE / FLOW

```
Does this value match any returned value?
```

You normally do not need `DISTINCT` merely for correctness.

The optimizer may already handle the existence semantics efficiently.

NOT IN with Subquery

`NOT IN` asks:

> Does this value fail to match every value returned by the subquery?

Example:

SQL

```sql
SELECT
    FundId,
    FundName

FROM fundraising.Fund

WHERE FundId NOT IN
(
    SELECT FundId
    FROM fundraising.GiftAllocation
);
```

This can work safely if:

EXAMPLE / FLOW

```
GiftAllocation.FundId
```

cannot be NULL.

NOT IN and NULL — Very Important

`NOT IN` is dangerous when the subquery can return `NULL`.

Microsoft specifically warns that NULL values in an `IN` or `NOT IN` subquery can produce `UNKNOWN` and unexpected results.

This is one of the most important interview and production SQL rules.

FundraisingDb NULL Example

`Gift.CampaignId` can be optional.

Suppose this query returns:

EXAMPLE / FLOW

```
200
201
NULL
```

SQL

```sql
SELECT CampaignId
FROM fundraising.Gift;
```

Now consider:

SQL

```sql
SELECT
    CampaignId,
    CampaignName

FROM fundraising.Campaign

WHERE CampaignId NOT IN
(
    SELECT CampaignId
```

```
    FROM fundraising.Gift
);
```

Because the subquery contains:

EXAMPLE / FLOW

```
NULL
```

the logic becomes problematic.

SQL Server cannot prove:

EXAMPLE / FLOW

```
CampaignId is not equal to NULL
```

because comparison with NULL is **UNKNOWN**.

The result may therefore be very different from what you intended.

Safer NOT IN Version

Filter NULL explicitly:

SQL

```
SELECT
    CampaignId,
    CampaignName

FROM fundraising.Campaign

WHERE CampaignId NOT IN
(
    SELECT CampaignId
    FROM fundraising.Gift
    WHERE CampaignId IS NOT NULL
);
```

Better still, when expressing anti-relationship logic, `NOT EXISTS` is often clearer.

EXISTS

Simple Definition

`EXISTS` asks:

> Does the subquery return at least one row?

If yes:

EXAMPLE / FLOW

```
TRUE
```

If no:

EXAMPLE / FLOW

```
FALSE
```

The actual selected value inside the EXISTS subquery does not matter; row existence is what matters. Microsoft notes that even `SELECT NULL` inside `EXISTS` evaluates true when a row exists.

EXISTS Example — Constituents Who Have Gifts

SQL

```
SELECT
    c.ConstituentId,
    c.DisplayName

FROM constituent.Constituent AS c
```

```
WHERE EXISTS
(
    SELECT 1

    FROM fundraising.Gift AS g

    WHERE g.ConstituentId =
          c.ConstituentId
);
```

EXISTS Flow

Outer row:

EXAMPLE / FLOW

```
Constituent 1001
```

Inner check:

EXAMPLE / FLOW

```
Is there at least one Gift
where ConstituentId = 1001?
```

If yes:

EXAMPLE / FLOW

```
keep Constituent 1001
```

Move to next Constituent.

Why SELECT 1?

You commonly see:

SQL

```
SELECT 1
```

inside `EXISTS`.

That means:

> I do not need any data from the subquery. I only care whether a row exists.

This would also logically work:

SQL

```
SELECT NULL
```

or:

SQL

```
SELECT *
```

because `EXISTS` cares only about row existence.

`SELECT 1` is simply a clear convention.

NOT EXISTS

Simple Definition

`NOT EXISTS` asks:

> Does the subquery return zero matching rows?

Example:

> Find Constituents who have never made a Gift.

SQL

```
SELECT
    c.ConstituentId,
    c.DisplayName

FROM constituent.Constituent AS c

WHERE NOT EXISTS
(
    SELECT 1

    FROM fundraising.Gift AS g

    WHERE g.ConstituentId =
          c.ConstituentId
);
```

NOT EXISTS Flow

EXAMPLE / FLOW

```
Constituent 1001
      |
      v
Matching Gift exists?
      |
      +--- YES → exclude
      |
      +--- NO  → include
```

Practical NOT EXISTS Example — Funds Never Used

SQL

```
SELECT
    f.FundId,
    f.FundName

FROM fundraising.Fund AS f

WHERE NOT EXISTS
(
    SELECT 1

    FROM fundraising.GiftAllocation AS ga

    WHERE ga.FundId =
          f.FundId
);
```

EXISTS vs IN

Both can sometimes solve the same requirement.

Example requirement:

> Constituents who have at least one Gift.

Using `IN`:

SQL

```
SELECT
    c.ConstituentId,
    c.DisplayName

FROM constituent.Constituent AS c

WHERE c.ConstituentId IN
(
    SELECT g.ConstituentId
```

```
    FROM fundraising.Gift AS g
);
```

Using **EXISTS**:

SQL

```
SELECT
    c.ConstituentId,
    c.DisplayName

FROM constituent.Constituent AS c

WHERE EXISTS
(
    SELECT 1

    FROM fundraising.Gift AS g

    WHERE g.ConstituentId =
          c.ConstituentId
);
```

Microsoft shows that **EXISTS** and **IN** can be semantically equivalent for some query shapes.

Which Is Better — EXISTS or IN?

Do not memorize:

EXAMPLE / FLOW

```
EXISTS is always faster.
```

That is not a reliable rule.

SQL Server's optimizer may transform both into very similar execution plans.

Choose primarily based on meaning.

Use **IN** when your requirement naturally sounds like:

EXAMPLE / FLOW

```
Is this value in this set?
```

Use **EXISTS** when it sounds like:

EXAMPLE / FLOW

```
Does a related row exist?
```

For correlated relationship checks, **EXISTS** is often clearer.

NOT EXISTS vs NOT IN

This distinction is more important.

NOT EXISTS usually expresses anti-relationship logic safely even when the related table contains nullable columns.

NOT IN can behave unexpectedly if the returned list contains **NULL**.

Recommended Pattern for Absence Checks

Instead of:

SQL

```
WHERE CampaignId NOT IN
(
    SELECT CampaignId
    FROM fundraising.Gift
)
```

prefer:

```sql
WHERE NOT EXISTS
(
    SELECT 1
    FROM fundraising.Gift AS g
    WHERE g.CampaignId =
          c.CampaignId
)
```

when expressing:

> No matching Gift exists for this Campaign.

ANY Simple Definition

ANY compares a value to a set returned by a single-column subquery.

The condition is true when the comparison succeeds against **at least one** returned value.

Syntax:

```sql
expression comparison_operator
ANY
(
    subquery
)
```

ANY Example

Requirement:

> Find Gifts larger than at least one Gift made to Campaign 200.

```sql
SELECT
    GiftId,
    CampaignId,
    TotalAmount

FROM fundraising.Gift

WHERE TotalAmount >
ANY
(
    SELECT TotalAmount

    FROM fundraising.Gift

    WHERE CampaignId = 200
);
```

Suppose Campaign 200 has Gifts:

```
$100
$500
$1,000
```

Then:

```
> ANY
```

means:

> Greater than at least one of those amounts.

So a Gift of:

EXAMPLE / FLOW

```
$600
```

qualifies because:

EXAMPLE / FLOW

```
$600 > $100
```

and:

EXAMPLE / FLOW

```
$600 > $500
```

It does not need to be greater than all values.

SOME

SOME is equivalent to:

EXAMPLE / FLOW

```
ANY
```

in SQL Server.

So:

SQL

```
> SOME (...)
```

and:

SQL

```
> ANY (...)
```

have the same meaning.

Most SQL developers use:

EXAMPLE / FLOW

```
ANY
```

more often.

ALL

Simple Definition

ALL requires the comparison to succeed against **every** value returned by the subquery.

Syntax:

SQL

```
expression comparison_operator
ALL
(
    subquery
)
```

ALL Example

Requirement:

> Find Gifts larger than every Gift in Campaign 200.

SQL

```
SELECT
    GiftId,
    TotalAmount

FROM fundraising.Gift
```

```
WHERE TotalAmount >
ALL
(
    SELECT TotalAmount

    FROM fundraising.Gift

    WHERE CampaignId = 200
);
```

Suppose Campaign 200 contains:

EXAMPLE / FLOW

```
$100
$500
$1,000
```

Then an outer Gift must be greater than:

EXAMPLE / FLOW

```
$100
AND
$500
AND
$1,000
```

Therefore:

EXAMPLE / FLOW

```
$1,500
```

qualifies.

EXAMPLE / FLOW

```
$600
```

does not.

ANY vs ALL

EXAMPLE / FLOW

```
> ANY
→ greater than at least one value
```

EXAMPLE / FLOW

```
> ALL
→ greater than every value
```

Example set:

EXAMPLE / FLOW

```
100
500
1000
```

Then:

EXAMPLE / FLOW

```
600 > ANY
→ TRUE
```

because 600 > 100.

But:

EXAMPLE / FLOW

```
600 > ALL
→ FALSE
```

because 600 is not greater than 1000.

ANY / ALL and MIN / MAX

Conceptually:

EXAMPLE / FLOW

```
> ANY
```

often resembles:

EXAMPLE / FLOW

```
> minimum value
```

and:

EXAMPLE / FLOW

```
> ALL
```

often resembles:

EXAMPLE / FLOW

```
> maximum value
```

Microsoft uses this intuition in its subquery documentation.

However, do not blindly replace `ANY` or `ALL` with `MIN` or `MAX` without considering:

EXAMPLE / FLOW

```
NULL values

empty subquery results

different comparison operators
```

because edge-case semantics can differ.

Comparison Operators with Subqueries

Common comparison operators include:

EXAMPLE / FLOW

```
=

<>

!=

>

>=

<

<=
```

Example:

SQL

```
WHERE TotalAmount >
(
    SELECT AVG(TotalAmount)
    FROM fundraising.Gift
)
```

The ordinary comparison form requires the subquery to return a single value.

Comparison Example — Largest Gift

SQL

```
SELECT
    GiftId,
    ConstituentId,
    TotalAmount

FROM fundraising.Gift

WHERE TotalAmount =
(
    SELECT MAX(TotalAmount)
    FROM fundraising.Gift
);
```

The subquery returns one scalar value:

EXAMPLE / FLOW

```
largest Gift amount
```

All Gifts equal to that amount are returned.

Comparison Example — Above Average Campaign Goal

SQL

```
SELECT
    CampaignId,
    CampaignName,
    GoalAmount

FROM fundraising.Campaign

WHERE GoalAmount >
(
    SELECT AVG(GoalAmount)
    FROM fundraising.Campaign
    WHERE GoalAmount IS NOT NULL
);
```

Subquery Returning One Value

These are common ways to guarantee a one-value result:

EXAMPLE / FLOW

```
MAX()

MIN()

AVG()

SUM()

COUNT()

Primary-key equality

Unique-key equality

TOP (1) with ORDER BY
```

Example:

SQL

```
SELECT COUNT(*)
FROM fundraising.Gift;
```

always produces one row containing the count.

Subquery Returning Multiple Rows

Use operators that support sets:

EXAMPLE / FLOW

```
IN

NOT IN

EXISTS

NOT EXISTS

ANY

SOME

ALL
```

Example:

SQL

```
WHERE CampaignId IN
(
    SELECT CampaignId
    FROM fundraising.Campaign
    WHERE Status = 'Active'
)
```

Subquery Returning Multiple Columns

Use it primarily as a table expression.

Example:

SQL

```
SELECT
    c.DisplayName,
    x.GiftCount,
    x.TotalGiving

FROM constituent.Constituent AS c

INNER JOIN
(
    SELECT
        ConstituentId,
        COUNT(*) AS GiftCount,
        SUM(TotalAmount) AS TotalGiving

    FROM fundraising.Gift

    GROUP BY ConstituentId
) AS x

    ON x.ConstituentId =
       c.ConstituentId;
```

Nested Subqueries

A subquery can contain another subquery.

Example business requirement:



Find Constituents who donated to Campaigns whose GoalAmount is greater than the average Campaign goal.

Start with average Campaign goal:

```sql
SELECT AVG(GoalAmount)
FROM fundraising.Campaign
WHERE GoalAmount IS NOT NULL;
```

Find Campaigns above average:

```sql
SELECT CampaignId
FROM fundraising.Campaign

WHERE GoalAmount >
(
    SELECT AVG(GoalAmount)
    FROM fundraising.Campaign
    WHERE GoalAmount IS NOT NULL
);
```

Then find Constituents who donated to those Campaigns:

```sql
SELECT
    ConstituentId,
    DisplayName

FROM constituent.Constituent

WHERE ConstituentId IN
(
    SELECT ConstituentId

    FROM fundraising.Gift

    WHERE CampaignId IN
    (
        SELECT CampaignId

        FROM fundraising.Campaign

        WHERE GoalAmount >
        (
            SELECT AVG(GoalAmount)

            FROM fundraising.Campaign

            WHERE GoalAmount IS NOT NULL
        )
    )
);
```

Nested Subquery Flow

EXAMPLE / FLOW

```
Level 3
Average Campaign Goal
        |
        v
Level 2
Campaigns above average
        |
        v
Level 1
Constituents who gave
to those Campaigns
        |
        v
Outer Query
Constituent details
```

How Deep Can Subqueries Be Nested?

SQL Server documentation states that subqueries may theoretically be nested up to:

EXAMPLE / FLOW

```
32 levels
```

although the practical limit can be lower depending on available memory and the complexity of the full statement.

That does **not** mean you should write 32-level subqueries.

Deep nesting quickly becomes difficult to:

EXAMPLE / FLOW

```
read

debug

test

maintain

tune
```

At that point, consider:

EXAMPLE / FLOW

```
CTEs

derived tables

temporary tables

views
```

depending on the requirement.

Correlated Subquery Example — Gifts Above Donor Average

Business requirement:

> Find Gifts larger than that same Constituent's own average Gift.

This is different from the overall average.

SQL

```sql
SELECT
    g.GiftId,
    g.ConstituentId,
    g.TotalAmount

FROM fundraising.Gift AS g

WHERE g.TotalAmount >
(
    SELECT
        AVG(g2.TotalAmount)

    FROM fundraising.Gift AS g2

    WHERE g2.ConstituentId =
          g.ConstituentId
);
```

Understand the Logic

For John:

EXAMPLE / FLOW

```
John's Gifts:

100
500
```

```
900

John average:
500
```

So John's outer Gifts above 500 are kept.

For Sarah:

EXAMPLE / FLOW

```
Sarah's Gifts:

2000
3000

Sarah average:
2500
```

Sarah is compared against Sarah's own average.

The subquery changes according to:

EXAMPLE / FLOW

```
current outer ConstituentId
```

Therefore it is correlated.

Another Correlated Example — Constituents with Large Gifts

SQL

```
SELECT
    c.ConstituentId,
    c.DisplayName

FROM constituent.Constituent AS c

WHERE EXISTS
(
    SELECT 1

    FROM fundraising.Gift AS g

    WHERE g.ConstituentId =
          c.ConstituentId

      AND g.TotalAmount >= 10000
);
```

This means:

Keep this Constituent if at least one related Gift of $10,000 or more exists.

Correlated Subquery in SELECT — Gift Allocation Difference

SQL

```
SELECT
    g.GiftId,
    g.TotalAmount,

    COALESCE
    (
        (
            SELECT
                SUM(ga.AllocationAmount)

            FROM fundraising.GiftAllocation AS ga

            WHERE ga.GiftId =
                  g.GiftId
```

```
        ),
        0
    ) AS AllocatedAmount,

    g.TotalAmount -
    COALESCE
    (
        (
            SELECT
                SUM(ga.AllocationAmount)

            FROM fundraising.GiftAllocation AS ga

            WHERE ga.GiftId =
                  g.GiftId
        ),
        0
    ) AS Difference

FROM fundraising.Gift AS g;
```

This works, but notice the same subquery is repeated twice.

That may not be ideal.

Better Rewrite with Pre-Aggregation

SQL

```
SELECT
    g.GiftId,
    g.TotalAmount,
    COALESCE(a.AllocatedAmount, 0)
        AS AllocatedAmount,

    g.TotalAmount -
    COALESCE(a.AllocatedAmount, 0)
        AS Difference

FROM fundraising.Gift AS g

LEFT JOIN
(
    SELECT
        GiftId,
        SUM(AllocationAmount)
            AS AllocatedAmount

    FROM fundraising.GiftAllocation

    GROUP BY GiftId
) AS a

    ON a.GiftId =
       g.GiftId;
```

Now the allocation aggregation is expressed once.

Converting Subqueries to JOINs

Many subqueries can be rewritten as joins.

That does not automatically mean the join is faster.

The optimizer may already transform them into equivalent physical operations.

Rewrite primarily when:

EXAMPLE / FLOW

```
the JOIN is clearer,

you need columns from the related table,

you want to pre-aggregate once,

or repeated correlated work is expensive.
```

IN to JOIN Example

Using `IN`:

SQL

```
SELECT
    c.ConstituentId,
    c.DisplayName

FROM constituent.Constituent AS c

WHERE c.ConstituentId IN
(
    SELECT
        g.ConstituentId

    FROM fundraising.Gift AS g
);
```

Possible JOIN version:

SQL

```
SELECT DISTINCT
    c.ConstituentId,
    c.DisplayName

FROM constituent.Constituent AS c

INNER JOIN fundraising.Gift AS g
    ON g.ConstituentId =
       c.ConstituentId;
```

Important JOIN Difference

Suppose John has:

EXAMPLE / FLOW

```
10 Gifts
```

The join naturally produces:

EXAMPLE / FLOW

```
10 John rows
```

before `DISTINCT`.

The `IN` query naturally expresses:

EXAMPLE / FLOW

```
Is John's ID present in Gift?
```

Therefore:

Do not rewrite to JOIN without understanding cardinality.

EXISTS to JOIN Example

EXISTS:

SQL

```
SELECT
    c.ConstituentId,
    c.DisplayName

FROM constituent.Constituent AS c

WHERE EXISTS
(
    SELECT 1
```

```
    FROM fundraising.Gift AS g

    WHERE g.ConstituentId =
          c.ConstituentId
);
```

JOIN:

SQL

```
SELECT DISTINCT
    c.ConstituentId,
    c.DisplayName

FROM constituent.Constituent AS c

INNER JOIN fundraising.Gift AS g
    ON g.ConstituentId =
       c.ConstituentId;
```

EXISTS often expresses the intent more directly because you only need to know:

EXAMPLE / FLOW

```
whether a Gift exists
```

rather than retrieving Gift data.

NOT EXISTS to LEFT JOIN

NOT EXISTS:

SQL

```
SELECT
    c.ConstituentId,
    c.DisplayName

FROM constituent.Constituent AS c

WHERE NOT EXISTS
(
    SELECT 1

    FROM fundraising.Gift AS g

    WHERE g.ConstituentId =
          c.ConstituentId
);
```

Equivalent anti-join pattern:

SQL

```
SELECT
    c.ConstituentId,
    c.DisplayName

FROM constituent.Constituent AS c

LEFT JOIN fundraising.Gift AS g
    ON g.ConstituentId =
       c.ConstituentId

WHERE g.GiftId IS NULL;
```

Use a right-side column such as:

EXAMPLE / FLOW

```
GiftId
```

that is guaranteed non-null for real matching rows.

NOT EXISTS vs LEFT JOIN Performance

Do not memorize:

EXAMPLE / FLOW

```
NOT EXISTS is always faster
```

or:

EXAMPLE / FLOW

```
LEFT JOIN is always faster.
```

SQL Server may optimize both into similar anti-semi-join strategies.

Choose the version that most clearly communicates the requirement, then inspect the execution plan if performance matters.

Subquery Performance Considerations

Subqueries are not inherently:

EXAMPLE / FLOW

```
slow
```

or:

EXAMPLE / FLOW

```
fast.
```

Performance depends on the full query.

Important factors include:

EXAMPLE / FLOW

```
indexes

row counts

statistics

cardinality

correlation predicates

aggregations

sorting

data types

filter selectivity

query shape

optimizer transformations
```

Performance Issue 1 — Correlated Subqueries

Example:

SQL

```sql
SELECT
    g.GiftId,

    (
        SELECT SUM(ga.AllocationAmount)

        FROM fundraising.GiftAllocation AS ga

        WHERE ga.GiftId =
              g.GiftId
    )

FROM fundraising.Gift AS g;
```

If `Gift` contains millions of rows and `GiftAllocation.GiftId` is not efficiently searchable, this can become expensive.

Useful indexing may include:

EXAMPLE / FLOW

```
GiftAllocation.GiftId
```

depending on the overall workload.

Performance Issue 2 — Repeated Scalar Subqueries

Avoid repeating the same expensive subquery several times.

Less ideal:

EXAMPLE / FLOW

```
Same SUM subquery
used 3 or 4 times
```

Consider:

EXAMPLE / FLOW

```
derived table

CTE

JOIN

APPLY

temporary table
```

depending on the requirement.

Performance Issue 3 — NOT IN with Nullable Results

This is partly correctness and partly query design.

If the subquery column is nullable:

EXAMPLE / FLOW

```
NOT IN
```

may produce unexpected **UNKNOWN** results.

Prefer:

EXAMPLE / FLOW

```
NOT EXISTS
```

for relationship anti-matching.

Performance Issue 4 — Unnecessary DISTINCT

Example:

SQL

```
WHERE ConstituentId IN
(
    SELECT DISTINCT ConstituentId
    FROM fundraising.Gift
)
```

The **DISTINCT** may be unnecessary because **IN** already tests membership rather than returning one result per duplicate.

Do not add duplicate-removal work without a reason.

Performance Issue 5 — Selecting Unnecessary Columns

In an existence test, prefer the intent:

```sql
SELECT 1
```

instead of thinking the subquery needs to return all columns.

The optimizer is generally smart enough to ignore unused values, but `SELECT 1` communicates the requirement clearly.

Performance Issue 6 — Deeply Nested Queries

A query such as:

```
Outer
  ↓
Subquery
  ↓
Subquery
  ↓
Subquery
  ↓
Subquery
```

may still optimize correctly, but it becomes harder for humans to maintain.

Consider a CTE when stages deserve names.

Performance Issue 7 — Implicit Conversion

Make sure correlated or comparison columns use compatible data types.

Example relationships such as:

```
Gift.ConstituentId
=
Constituent.ConstituentId
```

should use compatible types.

Implicit conversions can sometimes prevent efficient index usage.

Performance Issue 8 — Functions on Search Columns

Less ideal:

```sql
WHERE YEAR(g.GiftDate) IN
(
    SELECT YEAR(StartDate)
    FROM fundraising.Campaign
)
```

Functions on indexed search columns can reduce SARGability.

Prefer range predicates when appropriate.

Performance Issue 9 — Large IN Lists vs Subqueries

A subquery such as:

```sql
IN
(
    SELECT CampaignId
    FROM fundraising.Campaign
    WHERE ...
)
```

is usually clearer than constructing enormous literal lists in application code.

Microsoft also warns that extremely large explicit `IN (...)` constant lists can consume significant resources.

Performance Issue 10 — Check the Actual Plan

For important queries, compare:

EXAMPLE / FLOW

```
Actual execution plan

Logical reads

CPU

Elapsed time

Estimated vs actual rows
```

Do not tune based only on:

EXAMPLE / FLOW

```
"This uses a subquery"
```

or:

EXAMPLE / FLOW

```
"This uses a join."
```

Subquery Rules Summary

Important SQL Server rules include:

EXAMPLE / FLOW

```
A scalar comparison subquery must return one value.

IN / ANY / SOME / ALL subqueries return one column.

EXISTS checks only whether rows exist.

ORDER BY in a subquery requires TOP
or an allowed OFFSET/FETCH context.

IN / NOT IN with NULL requires care.

Derived tables in FROM require an alias.

Correlated subqueries reference outer-query columns.

Deep nesting is supported but should remain readable.
```

SQL Server's documented subquery restrictions include limits on `ORDER BY`, `INTO`, comparison-subquery column counts, and nested levels.

Practical Pattern 1 — Gifts Above Overall Average

SQL

```sql
SELECT
    GiftId,
    ConstituentId,
    TotalAmount

FROM fundraising.Gift

WHERE TotalAmount >
(
    SELECT AVG(TotalAmount)
    FROM fundraising.Gift
);
```

Type:

EXAMPLE / FLOW

```
Scalar

Non-correlated

WHERE subquery
```

Practical Pattern 2 — Gifts Above Donor's Own Average

SQL

```sql
SELECT
    g.GiftId,
    g.ConstituentId,
    g.TotalAmount

FROM fundraising.Gift AS g

WHERE g.TotalAmount >
(
    SELECT AVG(g2.TotalAmount)

    FROM fundraising.Gift AS g2

    WHERE g2.ConstituentId =
          g.ConstituentId
);
```

Type:

EXAMPLE / FLOW

```
Scalar

Correlated

WHERE subquery
```

Practical Pattern 3 — Constituents Who Have Gifts

SQL

```sql
SELECT
    c.ConstituentId,
    c.DisplayName

FROM constituent.Constituent AS c

WHERE EXISTS
(
    SELECT 1

    FROM fundraising.Gift AS g

    WHERE g.ConstituentId =
          c.ConstituentId
);
```

Pattern:

EXAMPLE / FLOW

```
EXISTS
```

Practical Pattern 4 — Constituents Without Gifts

SQL

```sql
SELECT
    c.ConstituentId,
    c.DisplayName

FROM constituent.Constituent AS c
```

```
WHERE NOT EXISTS
(
    SELECT 1

    FROM fundraising.Gift AS g

    WHERE g.ConstituentId =
          c.ConstituentId
);
```

Pattern:

EXAMPLE / FLOW

NOT EXISTS

Practical Pattern 5 — Gifts in Active Campaigns

SQL

```
SELECT
    GiftId,
    CampaignId,
    TotalAmount

FROM fundraising.Gift

WHERE CampaignId IN
(
    SELECT CampaignId

    FROM fundraising.Campaign

    WHERE Status = 'Active'
);
```

Pattern:

EXAMPLE / FLOW

IN + multi-row subquery

Practical Pattern 6 — Funds Never Used

SQL

```
SELECT
    f.FundId,
    f.FundName

FROM fundraising.Fund AS f

WHERE NOT EXISTS
(
    SELECT 1

    FROM fundraising.GiftAllocation AS ga

    WHERE ga.FundId =
          f.FundId
);
```

Practical Pattern 7 — Allocation Summary Derived Table

SQL

```
SELECT
    g.GiftId,
    g.TotalAmount,
    a.AllocatedAmount

FROM fundraising.Gift AS g
```

```
LEFT JOIN
(
    SELECT
        GiftId,
        SUM(AllocationAmount)
            AS AllocatedAmount

    FROM fundraising.GiftAllocation

    GROUP BY GiftId
) AS a

    ON a.GiftId =
       g.GiftId;
```

Pattern:

EXAMPLE / FLOW

```
FROM subquery
+
JOIN
```

Practical Pattern 8 — Latest Interaction Date

SQL

```
SELECT
    c.ConstituentId,
    c.DisplayName,

    (
        SELECT MAX(i.InteractionDate)

        FROM engagement.Interaction AS i

        WHERE i.ConstituentId =
              c.ConstituentId
    ) AS LatestInteractionDate

FROM constituent.Constituent AS c;
```

Practical Pattern 9 — Constituents Assigned to Same User Who Interacted

SQL

```
SELECT
    t.TaskId,
    t.ConstituentId,
    t.AssignedToUserId

FROM engagement.Task AS t

WHERE EXISTS
(
    SELECT 1

    FROM engagement.Interaction AS i

    WHERE i.ConstituentId =
          t.ConstituentId

      AND i.FundraiserUserId =
          t.AssignedToUserId
);
```

This is an excellent multi-column relationship comparison pattern.

Practical Pattern 10 — Campaigns Above Average Goal

```sql
SELECT
    CampaignId,
    CampaignName,
    GoalAmount

FROM fundraising.Campaign

WHERE GoalAmount >
(
    SELECT AVG(GoalAmount)

    FROM fundraising.Campaign

    WHERE GoalAmount IS NOT NULL
);
```

Practical Pattern 11 — Gift Greater Than Any Campaign-200 Gift

```sql
SELECT
    GiftId,
    TotalAmount

FROM fundraising.Gift

WHERE TotalAmount >
ANY
(
    SELECT TotalAmount

    FROM fundraising.Gift

    WHERE CampaignId = 200
);
```

Practical Pattern 12 — Gift Greater Than All Campaign-200 Gifts

```sql
SELECT
    GiftId,
    TotalAmount

FROM fundraising.Gift

WHERE TotalAmount >
ALL
(
    SELECT TotalAmount

    FROM fundraising.Gift

    WHERE CampaignId = 200
);
```

Subquery vs CTE

A subquery is useful when logic is:

```
small

local

used once
```

A CTE is often better when:

EXAMPLE / FLOW

```
multiple logical stages exist

the subquery becomes deeply nested

the same logical intermediate result
deserves a meaningful name

recursive processing is needed
```

Example:

EXAMPLE / FLOW

```
Simple average lookup
→ Subquery
```

EXAMPLE / FLOW

```
Aggregate donors
then rank them
then join interaction summary
→ CTE
```

Subquery vs Derived Table

Technically:

EXAMPLE / FLOW

```
a derived table is a subquery in FROM.
```

Use it when you need:

EXAMPLE / FLOW

```
multiple columns

multiple rows

a temporary table-like result
inside one statement.
```

Subquery vs JOIN

Use a subquery when the requirement naturally means:

EXAMPLE / FLOW

```
compare against a value

check membership

check existence

check absence
```

Use JOIN when:

EXAMPLE / FLOW

```
you need columns from both related sources

you need to combine relational data directly.
```

Subquery vs EXISTS

EXISTS is itself a subquery pattern.

It is especially appropriate when the business question is:

EXAMPLE / FLOW

```
Does at least one matching row exist?
```

Example:

EXAMPLE / FLOW

```
Does this Constituent have a Gift?
```

Common Mistakes

Mistake 1 — Using = with Multiple Rows

Wrong:

SQL

```
WHERE CampaignId =
(
    SELECT CampaignId
    FROM fundraising.Campaign
    WHERE Status = 'Active'
)
```

if multiple Campaigns are active.

Use:

SQL

```
IN
```

instead.

Mistake 2 — Forgetting NOT IN NULL Behavior

Dangerous:

SQL

```
WHERE CampaignId NOT IN
(
    SELECT CampaignId
    FROM fundraising.Gift
)
```

if the subquery can return **NULL**.

Prefer:

EXAMPLE / FLOW

```
NOT EXISTS
```

or filter NULL explicitly.

Mistake 3 — Using Correlated Subquery Without Index Support

Example correlation:

EXAMPLE / FLOW

```
GiftAllocation.GiftId
=
Gift.GiftId
```

may need appropriate indexing when heavily used.

Mistake 4 — Returning Extra Columns in Scalar Subquery

Wrong concept:

SQL

```
WHERE GiftId =
(
    SELECT GiftId, TotalAmount
    FROM fundraising.Gift
)
```

An ordinary scalar comparison expects one expression.

Mistake 5 — Deep Nesting Without Need

If a query becomes:

EXAMPLE / FLOW

```
four or five deeply nested levels
```

consider whether CTEs would make it clearer.

Mistake 6 — Converting EXISTS to JOIN and Creating Duplicates

One Constituent with ten Gifts becomes ten rows after a join.

Do not add:

EXAMPLE / FLOW

```
DISTINCT
```

blindly just to hide cardinality problems.

Mistake 7 — Assuming EXISTS Is Always Faster Than IN

The optimizer can often produce similar strategies.

Choose the form that best represents the logic.

Mistake 8 — Assuming JOIN Is Always Faster Than Subquery

Again, query form alone does not determine performance.

Measure the execution plan.

Mistake 9 — TOP 1 Without ORDER BY

This:

SQL

```
SELECT TOP (1)
    GiftId
FROM fundraising.Gift;
```

does not specify which Gift should be chosen.

Use deterministic:

SQL

```
ORDER BY
    GiftDate DESC,
    GiftId DESC
```

when asking for the latest Gift.

Mistake 10 — Repeating the Same Expensive Scalar Subquery

Calculate once using:

EXAMPLE / FLOW

```
derived table

CTE

APPLY

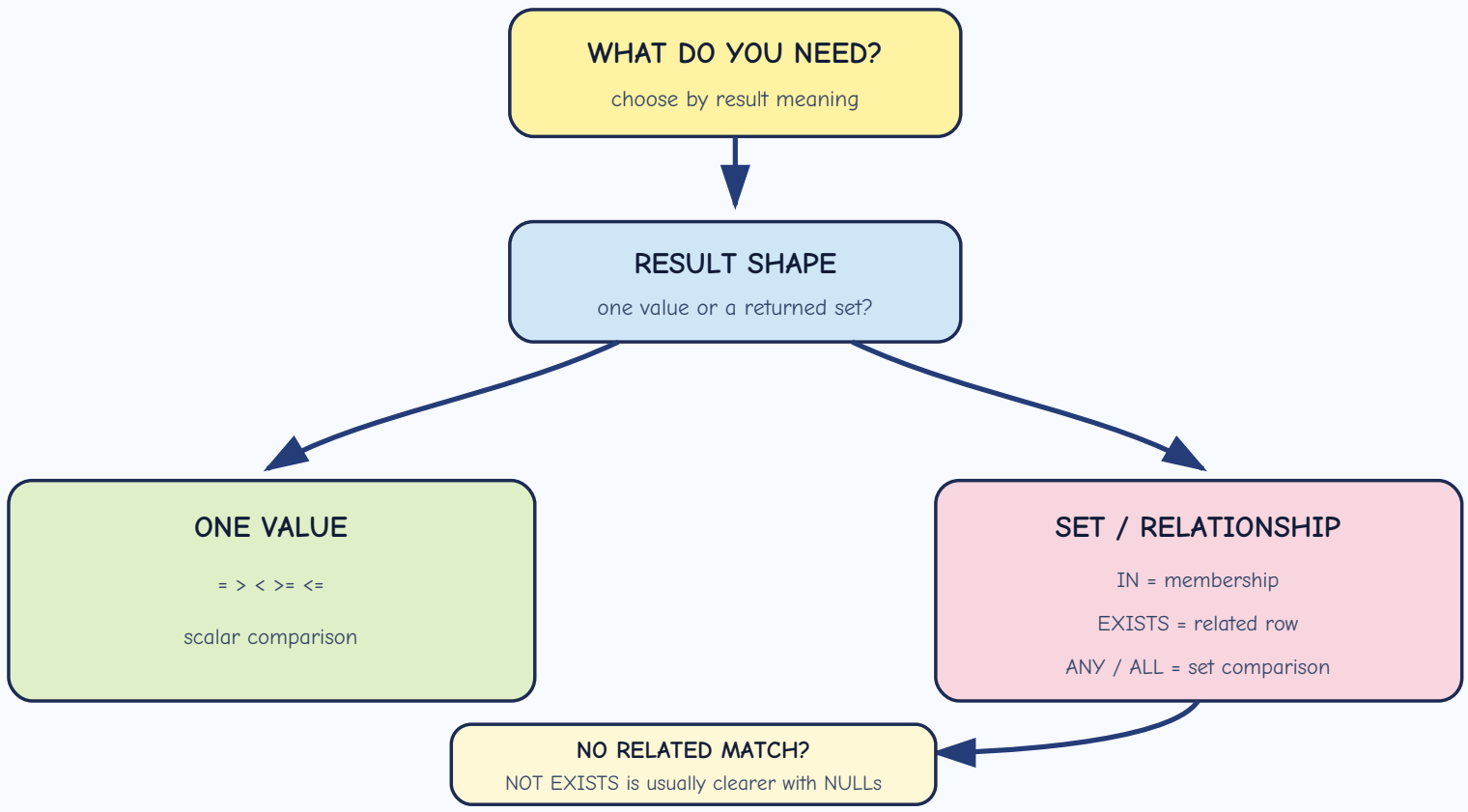
or temp table
```

when appropriate.

Master Subquery Decision Guide

3D DIAGRAM

Choosing the Subquery Operator



The operator is a semantic choice: scalar comparison, membership, existence, absence, or comparison against some/all returned values.

EXAMPLE / FLOW

```
Need one calculated value?
→ Scalar subquery
```

EXAMPLE / FLOW

```
Need to test membership in a returned list?
→ IN
```

EXAMPLE / FLOW

```
Need to know whether a related row exists?
→ EXISTS
```

EXAMPLE / FLOW

```
Need rows with no related match?
→ NOT EXISTS
```

EXAMPLE / FLOW

```
Need compare against at least one returned value?
→ ANY / SOME
```

EXAMPLE / FLOW

```
Need compare against every returned value?
→ ALL
```

EXAMPLE / FLOW

```
Need multiple rows and columns?
→ Derived table in FROM
```

EXAMPLE / FLOW

```
Need values based on each outer row?
→ Correlated subquery
```

EXAMPLE / FLOW

```
Need several named processing stages?
→ Consider a CTE
```

Master Classification Table

Type	Result	Fundraising Example
Scalar	One value	Average Gift
Single-row	One expected row	CampaignId by unique CampaignCode
Multi-row	Many rows, usually one column	Active CampaignIds
Multi-column	Several columns	Constituent giving summary
Non-correlated	Independent	Overall average Gift
Correlated	Uses outer row	Allocation total per Gift

Location Summary

Location	Typical Use
`SELECT`	Scalar lookup/calculation
`FROM`	Derived table
`WHERE`	Filtering, IN, EXISTS, comparisons
`HAVING`	Compare grouped results
`JOIN`	Join to derived result
`ON`	Comparison with a scalar subquery
DML	Filter/derive values for INSERT/UPDATE/DELETE

Operator Summary

EXAMPLE / FLOW

```
=
→ one expected value
```

EXAMPLE / FLOW

```
IN
→ value appears in set
```

EXAMPLE / FLOW

```
NOT IN
→ value not in set
→ beware NULL
```

EXAMPLE / FLOW

```
EXISTS
→ at least one matching row exists
```

EXAMPLE / FLOW

```
NOT EXISTS
→ no matching row exists
```

EXAMPLE / FLOW

```
ANY / SOME
→ comparison true for at least one returned value
```

EXAMPLE / FLOW

```
ALL
→ comparison true for every returned value
```

25 Interview Questions and Query Problems

1. What is a subquery?

A subquery is a SQL query written inside another SQL statement.

The inner query returns information used by the outer query.

Example:

SQL

```
SELECT
    GiftId,
    TotalAmount

FROM fundraising.Gift

WHERE TotalAmount >
(
    SELECT AVG(TotalAmount)
    FROM fundraising.Gift
);
```

The inner query calculates the average Gift amount.

The outer query returns Gifts above that average.

2. What is the difference between an inner query and an outer query?

The inner query is the subquery.

The outer query contains it and uses its result.

Example:

EXAMPLE / FLOW

```
Inner
→ Calculate average Gift.

Outer
→ Compare Gift.TotalAmount
  against that average.
```

3. What is a scalar subquery?

A scalar subquery returns one value.

Example:

SQL

```
SELECT
    GiftId,
    TotalAmount,

    (
        SELECT AVG(TotalAmount)
        FROM fundraising.Gift
    ) AS AverageGift

FROM fundraising.Gift;
```

4. What happens if a scalar comparison subquery returns more than one row?

SQL Server raises an error because operators such as:

EXAMPLE / FLOW

```
=
>
<
```

without **ANY** or **ALL** require a single value.

Use operators such as:

```
EXAMPLE / FLOW
IN

ANY

ALL
```

when multiple values are valid.

5. What is a multi-row subquery?

A multi-row subquery returns several rows.

Example:

```sql
SQL
SELECT CampaignId
FROM fundraising.Campaign
WHERE Status = 'Active';
```

It can be used with:

```
EXAMPLE / FLOW
IN

ANY

ALL
```

or as part of existence logic.

6. What is a correlated subquery?

A correlated subquery references a column from the outer query.

Example:

```sql
SQL
SELECT
    g.GiftId,
    g.TotalAmount

FROM fundraising.Gift AS g

WHERE g.TotalAmount >
(
    SELECT AVG(g2.TotalAmount)

    FROM fundraising.Gift AS g2

    WHERE g2.ConstituentId =
          g.ConstituentId
);
```

Each Gift is compared with its own Constituent's average Gift amount.

7. What is a non-correlated subquery?

A non-correlated subquery does not depend on the outer row.

Example:

```sql
SQL
WHERE TotalAmount >
(
    SELECT AVG(TotalAmount)
    FROM fundraising.Gift
)
```

The overall average is independent of the current Gift.

8. What is the difference between correlated and non-correlated subqueries?

A non-correlated subquery can conceptually execute independently.

A correlated subquery references the current outer row, so its logical result can change from one outer row to another.

9. Can a subquery be used in SELECT?

Yes, if it returns one scalar value.

Example:

SQL

```
SELECT
    c.ConstituentId,

    (
        SELECT MAX(i.InteractionDate)

        FROM engagement.Interaction AS i

        WHERE i.ConstituentId =
              c.ConstituentId
    ) AS LatestInteractionDate

FROM constituent.Constituent AS c;
```

10. What is a subquery in FROM called?

It is commonly called a:

EXAMPLE / FLOW

derived table.

Example:

SQL

```
SELECT *
FROM
(
    SELECT
        ConstituentId,
        SUM(TotalAmount) AS TotalGiving

    FROM fundraising.Gift

    GROUP BY ConstituentId
) AS x;
```

A derived table requires an alias.

11. What does IN do with a subquery?

`IN` checks whether a value matches any value returned by a single-column subquery.

Example:

SQL

```
SELECT *
FROM fundraising.Gift

WHERE CampaignId IN
(
    SELECT CampaignId
    FROM fundraising.Campaign
    WHERE Status = 'Active'
);
```

12. Why is NOT IN dangerous with NULL?

If the subquery returns NULL, comparisons can become **UNKNOWN**, producing unexpected results.

For anti-match logic, `NOT EXISTS` is often safer.

13. What does EXISTS do?

`EXISTS` returns true if the subquery returns at least one row.

Example:

SQL

```
SELECT
    c.ConstituentId,
    c.DisplayName

FROM constituent.Constituent AS c

WHERE EXISTS
(
    SELECT 1
    FROM fundraising.Gift AS g
    WHERE g.ConstituentId =
          c.ConstituentId
);
```

14. Why do we write SELECT 1 inside EXISTS?

Because the actual selected value is irrelevant.

`EXISTS` only checks whether a row exists.

`SELECT 1` communicates that intent clearly.

15. What is NOT EXISTS?

`NOT EXISTS` returns true when the subquery returns no matching rows.

Example:

SQL

```
SELECT
    f.FundId,
    f.FundName

FROM fundraising.Fund AS f

WHERE NOT EXISTS
(
    SELECT 1

    FROM fundraising.GiftAllocation AS ga

    WHERE ga.FundId =
          f.FundId
);
```

This finds Funds with no allocations.

16. EXISTS vs IN — which is faster?

Neither is universally faster.

For many equivalent queries, SQL Server can optimize both to similar execution strategies.

Choose based on the meaning:

EXAMPLE / FLOW

```
IN
→ value in set

EXISTS
→ related row exists
```

Then measure if performance matters.

17. NOT EXISTS vs NOT IN — which should I prefer?

For anti-relationship checks, `NOT EXISTS` is generally safer because `NOT IN` can produce surprising results when the subquery contains NULL.

18. What is ANY?

`ANY` means the comparison must be true for at least one value returned by the subquery.

Example:

SQL

```
WHERE TotalAmount >
ANY
(
    SELECT TotalAmount
    FROM fundraising.Gift
    WHERE CampaignId = 200
)
```

means greater than at least one Gift in Campaign 200.

19. What is SOME?

`SOME` is equivalent to `ANY` in SQL Server.

So:

EXAMPLE / FLOW

```
> SOME
```

means the same as:

EXAMPLE / FLOW

```
> ANY.
```

20. What is ALL?

`ALL` requires the comparison to succeed against every value returned by the subquery.

Example:

SQL

```
WHERE TotalAmount >
ALL
(
    SELECT TotalAmount
    FROM fundraising.Gift
    WHERE CampaignId = 200
)
```

means greater than every Gift amount in Campaign 200.

21. Query Problem — Find Gifts Above the Donor's Average Gift

SQL

```
SELECT
    g.GiftId,
    g.ConstituentId,
    g.TotalAmount

FROM fundraising.Gift AS g

WHERE g.TotalAmount >
(
    SELECT
        AVG(g2.TotalAmount)

    FROM fundraising.Gift AS g2

    WHERE g2.ConstituentId =
          g.ConstituentId
);
```

This is a correlated scalar subquery.

22. Query Problem — Find Constituents with Gifts but No Interactions

SQL

```
SELECT
    c.ConstituentId,
    c.DisplayName

FROM constituent.Constituent AS c

WHERE EXISTS
(
    SELECT 1

    FROM fundraising.Gift AS g

    WHERE g.ConstituentId =
          c.ConstituentId
)

AND NOT EXISTS
(
    SELECT 1

    FROM engagement.Interaction AS i

    WHERE i.ConstituentId =
          c.ConstituentId
);
```

Meaning:

EXAMPLE / FLOW

```
Gift exists
AND
Interaction does not exist.
```

23. Query Problem — Return Gift Allocation Totals

SQL

```
SELECT
    g.GiftId,
    g.TotalAmount,
    x.AllocatedAmount

FROM fundraising.Gift AS g

LEFT JOIN
(
    SELECT
        GiftId,
        SUM(AllocationAmount)
            AS AllocatedAmount

    FROM fundraising.GiftAllocation

    GROUP BY GiftId
) AS x

    ON x.GiftId =
       g.GiftId;
```

The subquery is a multi-row, multi-column derived table.

24. Query Problem — Find Campaigns with No Gifts

Recommended:

SQL

```
SELECT
    c.CampaignId,
    c.CampaignName

FROM fundraising.Campaign AS c
```

```
WHERE NOT EXISTS
(
    SELECT 1

    FROM fundraising.Gift AS g

    WHERE g.CampaignId =
          c.CampaignId
);
```

This avoids the NULL problem that can occur with `NOT IN` when `Gift.CampaignId` is nullable.

25. Full Interview Problem

Requirement

Return Constituents who:

EXAMPLE / FLOW

```
have at least one Gift,

have lifetime giving above
the overall average Constituent lifetime giving,

and have no open Task.
```

Step 1 — Constituent Giving Summary

SQL

```
SELECT
    ConstituentId,
    SUM(TotalAmount)
        AS LifetimeGiving

FROM fundraising.Gift

GROUP BY ConstituentId;
```

Step 2 — Calculate Average Lifetime Giving

SQL

```
SELECT
    AVG(x.LifetimeGiving)

FROM
(
    SELECT
        ConstituentId,

        SUM(TotalAmount)
            AS LifetimeGiving

    FROM fundraising.Gift

    GROUP BY ConstituentId
) AS x;
```

Step 3 — Complete Query

SQL

```
SELECT
    c.ConstituentId,
    c.DisplayName,
    gs.LifetimeGiving

FROM constituent.Constituent AS c
```

```
INNER JOIN
(
    SELECT
        ConstituentId,

        SUM(TotalAmount)
            AS LifetimeGiving

    FROM fundraising.Gift

    GROUP BY ConstituentId
) AS gs

    ON gs.ConstituentId =
       c.ConstituentId

WHERE gs.LifetimeGiving >
(
    SELECT
        AVG(x.LifetimeGiving)

    FROM
    (
        SELECT
            ConstituentId,

            SUM(TotalAmount)
                AS LifetimeGiving

        FROM fundraising.Gift

        GROUP BY ConstituentId
    ) AS x
)

AND NOT EXISTS
(
    SELECT 1

    FROM engagement.Task AS t

    WHERE t.ConstituentId =
          c.ConstituentId

      AND t.CompletedAtUtc IS NULL
);
```

Understand the Complete Problem

Derived subquery:

EXAMPLE / FLOW

```
Calculate lifetime giving
for each Constituent.
```

Scalar subquery:

EXAMPLE / FLOW

```
Calculate average lifetime giving
across those Constituents.
```

Outer query:

EXAMPLE / FLOW

```
Keep Constituents
above that average.
```

NOT EXISTS:

EXAMPLE / FLOW

```
Remove Constituents
who have an open Task.
```

This combines:

EXAMPLE / FLOW

```
derived tables

scalar subqueries

aggregation

JOIN

NOT EXISTS

correlation
```

in one realistic FundraisingDb query.

Final Revision Map

EXAMPLE / FLOW

```
SUBQUERY
   |
   +-----------------------------------+
   |                                   |
BY RESULT                         BY RELATION
   |                                   |
   +---- Scalar                        +---- Non-correlated
   |
   +---- Single-row                    +---- Correlated
   |
   +---- Multi-row
   |
   +---- Multi-column
```

Locations:

EXAMPLE / FLOW

```
SELECT
→ scalar calculated value

FROM
→ derived table

WHERE
→ filtering/comparison

HAVING
→ grouped-result filtering

JOIN
→ join to derived result
```

Operators:

EXAMPLE / FLOW

```
=
→ one value

IN
→ any matching value

NOT IN
→ no matching value
  but beware NULL

EXISTS
→ matching row exists
```

```
NOT EXISTS
→ matching row does not exist

ANY / SOME
→ comparison true for at least one

ALL
→ comparison true for every value
```

Performance:

EXAMPLE / FLOW

```
Correlated subqueries
→ check indexes and execution plan

Repeated scalar subqueries
→ consider JOIN / CTE / derived table

NOT IN
→ watch NULL

Deep nesting
→ consider CTEs

JOIN rewrites
→ understand duplicate/cardinality effects

Performance choice
→ measure, don't guess
```

Final Easy Memory Rules

EXAMPLE / FLOW

```
Subquery
→ Query inside another query
```

EXAMPLE / FLOW

```
Scalar subquery
→ One value
```

EXAMPLE / FLOW

```
IN
→ Is this value in that result set?
```

EXAMPLE / FLOW

```
EXISTS
→ Does at least one related row exist?
```

EXAMPLE / FLOW

```
NOT EXISTS
→ Does no related row exist?
```

EXAMPLE / FLOW

```
ANY
→ At least one comparison succeeds
```

EXAMPLE / FLOW

```
ALL
→ Every comparison succeeds
```

EXAMPLE / FLOW

```
Correlated
→ Inner query uses outer-row value
```

EXAMPLE / FLOW

```
Non-correlated
→ Inner query is independent
```

The most important rule from this chapter is:



First ask what the inner query needs to return—one value, many values, existence, or a table-shaped result. Then choose the correct subquery location and operator. Do not choose =, **IN**, **EXISTS**, **ANY**, or **ALL** until you understand the shape and meaning of the subquery result.